Lab 4: Implementing Nested Procedures and Sorting

Task 1

Translate the C function shown in Listing 1 into RISC-V assembly to calculate the factorial using the iterative approach. How does the stack get populated? Observe and comment on the working of code.

```
long long int fact (long long int n){
    if (n < 1)
        return (1);
    else return (n * fact(n - 1));
    }
```

Listing 1

ANS) The stack does not get populated because the iterative solution uses a loop instead of function calls, so we dont need to save return addresses on the stack.

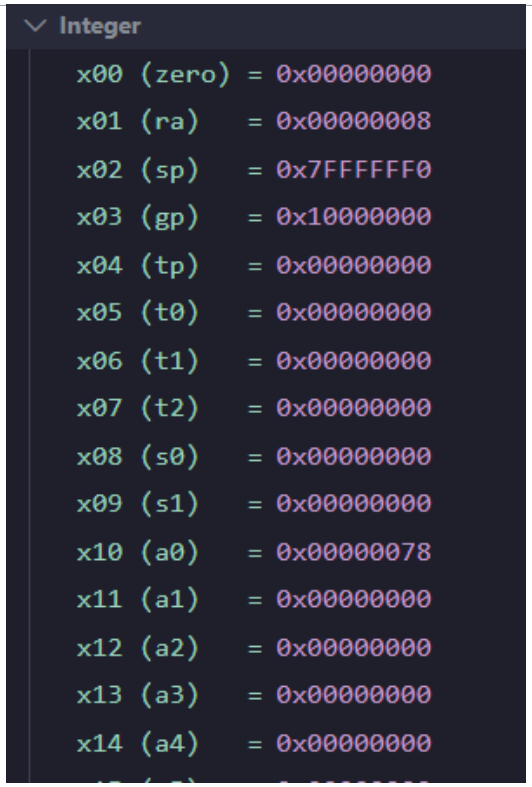
Code:

task1.s task2.s test.s

task1.s

```
.globl main
.text
main:
    addi x10, x0, 5   # n = 5
    jal x1, factorial

    addi x11, x10, 0
    addi x10, x0, 1
    ecall

    addi x10, x0, 10
    ecall

factorial:
    addi x5, x10, 0     # Copy n to x5
    addi x10, x0, 1      # reset x10 to 1

loop:
    ble x5, x0, done    # if iterator <= 0, finish
    mul x10, x10, x5     # result = result* value of n
    addi x5, x5, -1     # decrement n for next call
    jal x0, loop        # loop again

done:
    jalr x0, 0(x1)    # return to function caller
```

Output




```
Starting program C:\Users\HP\Desktop\4th Sem\CA\Lab 4\task1.s

120
```

Task 2

The nth triangular number is like a factorial but instead of a product we use a summation: https://en.wikipedia.org/wiki/Triangular_number

```
int ntri (int num){
  if (num<=1)
    return 1;

  return num + ntri (num-1);
}
```

Listing 5: Nth Triangular number

Translate the above C function to calculate the Nth Triangular number into RISC-V assembly. You should also write the wrapper code, i.e., which calls this function with an argument and then displays the result returned by this function.

Since there's recursion going on, you will need to adjust the stack so that you store the return 4 address as well as any useful variables of the caller on the stack before calling the callee. Show the result computed. How does the stack get populated? Observe and comment on the working of code.

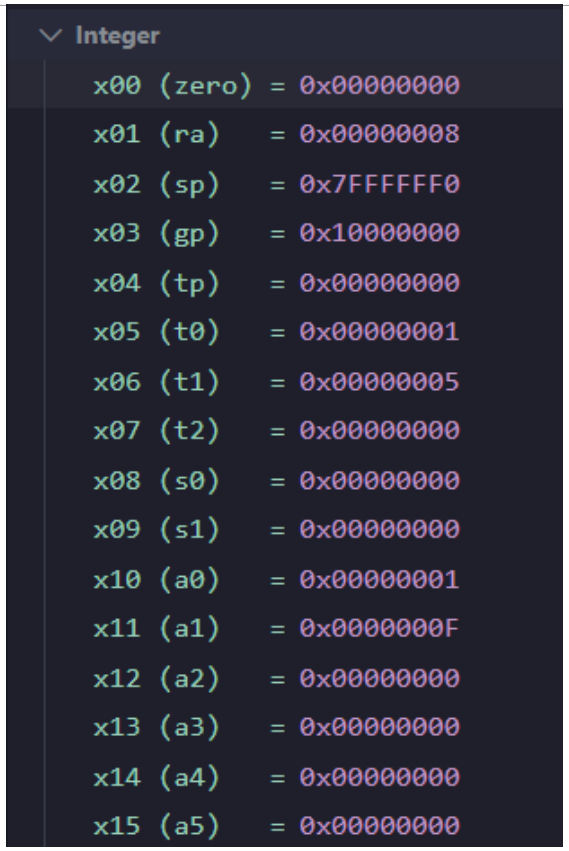
Code:

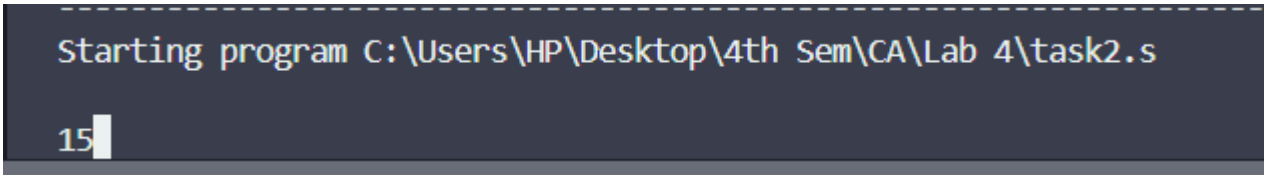
task1.s task2.s test.s

task2.s

```
.globl main
.text

main:
    addi x10, x0, 5  # Set argument n = 5
    jal x1, ntri        # Call the function

    addi x11, x10, 0   # result to x11
    addi x10, x0, 1      # to printe integer
    ecall
    addi x10, x0, 10   # Exit
    ecall

ntri:
    addi x2, x2, -8
    sw x1, 4(x2)
    sw x10, 0(x2)

    addi x5, x0, 1    # x5 = 1
    ble x10, x5, baseCase

    addi x10, x10, -1    # n = n - 1
    jal x1, ntri     # Recursive Call

    lw x6, 0(x2)
    add x10, x10, x6
    jal x0, exit_ntri

baseCase:
    addi x10, x0, 1  # Return 1

exit_ntri:
    lw x1, 4(x2)
    addi x2, x2, 8
    jalr x0, 0(x1)
```

Output





Task 3- Bubble Sort

```
void bubble (int *a, unsigned int len){

  if (a==NULL || len==0)
    return;

  for(int i=0; i<len; i++){
    for( int j=i; j<len; j++){
      if (a[i] < a[j]){
        int temp = a[i];
        a[i] = a[j];
        a[j] = temp;
      }
    }
  }

  return;
}
```

Listing 6

Write the equivalent RISC-V assembly code for Listing 6. a and len get passed in x10 and x11 respectively.

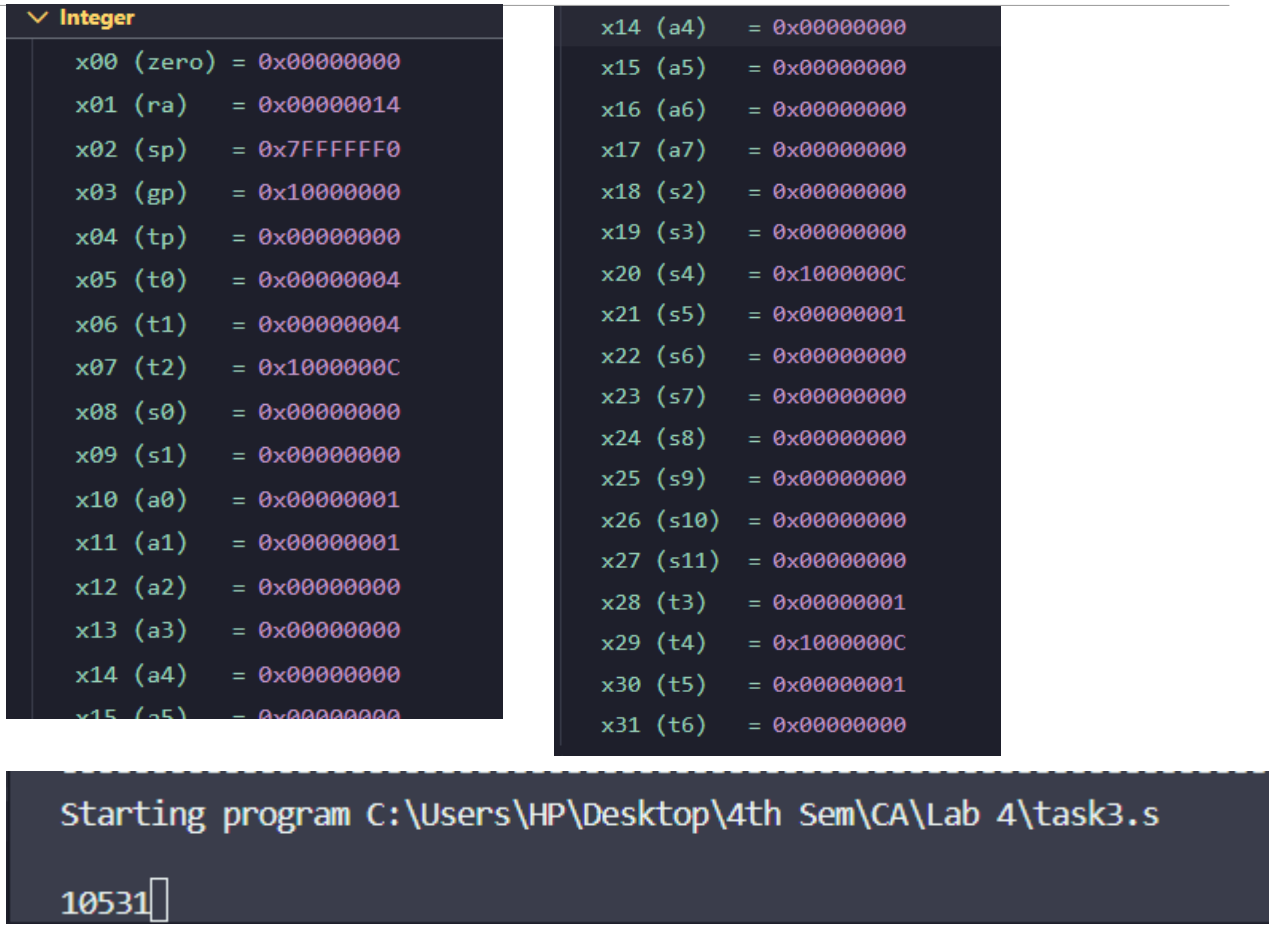
Code:

task3.s

```
.globl main
.data
    arr: .word 3, 10, 1, 5
    len: .word 4

.text
main:
    la x10, arr
    lw x11, len

    jal x1, bubble


    la x20, arr
    lw x21, len

print_loop:
    beq x21, x0, exit_prog # If counter is 0 then exit

    lw x11, 0(x20)
    addi x10, x0, 1     #Print Integer
    ecall

    addi x20, x20, 4     # offset
    addi x21, x21, -1    # counter - 1
    jal x0, print_loop

exit_prog:
    addi x10, x0, 10
    ecall
```

```
31
32  bubble:
33      beq x10, x0, return_func  # if a == NULL, return
34      beq x11, x0, return_func  # if len == 0, return
35
36      addi x5, x0, 0  # i = 0
37
38  outer_loop:
39      bge x5, x11, return_func  # if i >= len, return
40      addi x6, x5, 0        # j = i
41
42  inner_loop:
43      bge x6, x11, next_outer   # if j >= len, go to next outer loop
44      slli x7, x5, 2        # offset
45      add x7, x10, x7
46      lw x28, 0(x7)         # x28 = a[i]
47
48      slli x29, x6, 2       # offset
49      add x29, x10, x29      #
50      lw x30, 0(x29)       # x30 = a[j]
51      bge x28, x30, no_swap
52
53      sw x30, 0(x7)      #swap
54      sw x28, 0(x29)     #swap
55
56  no_swap:
57      addi x6, x6, 1        # j++
58      jal x0, inner_loop
59
60  next_outer:
61      addi x5, x5, 1        # i++
62      jal x0, outer_loop
63
64  return_func:
65      jalr x0, 0(x1)  # Return
```

Output



Task 4 - Student-Selected Program

In this task, students will design and implement a RISC-V assembly program of their own choice that demonstrates the concepts covered in this lab, such as nested procedures, stack management, and control flow.

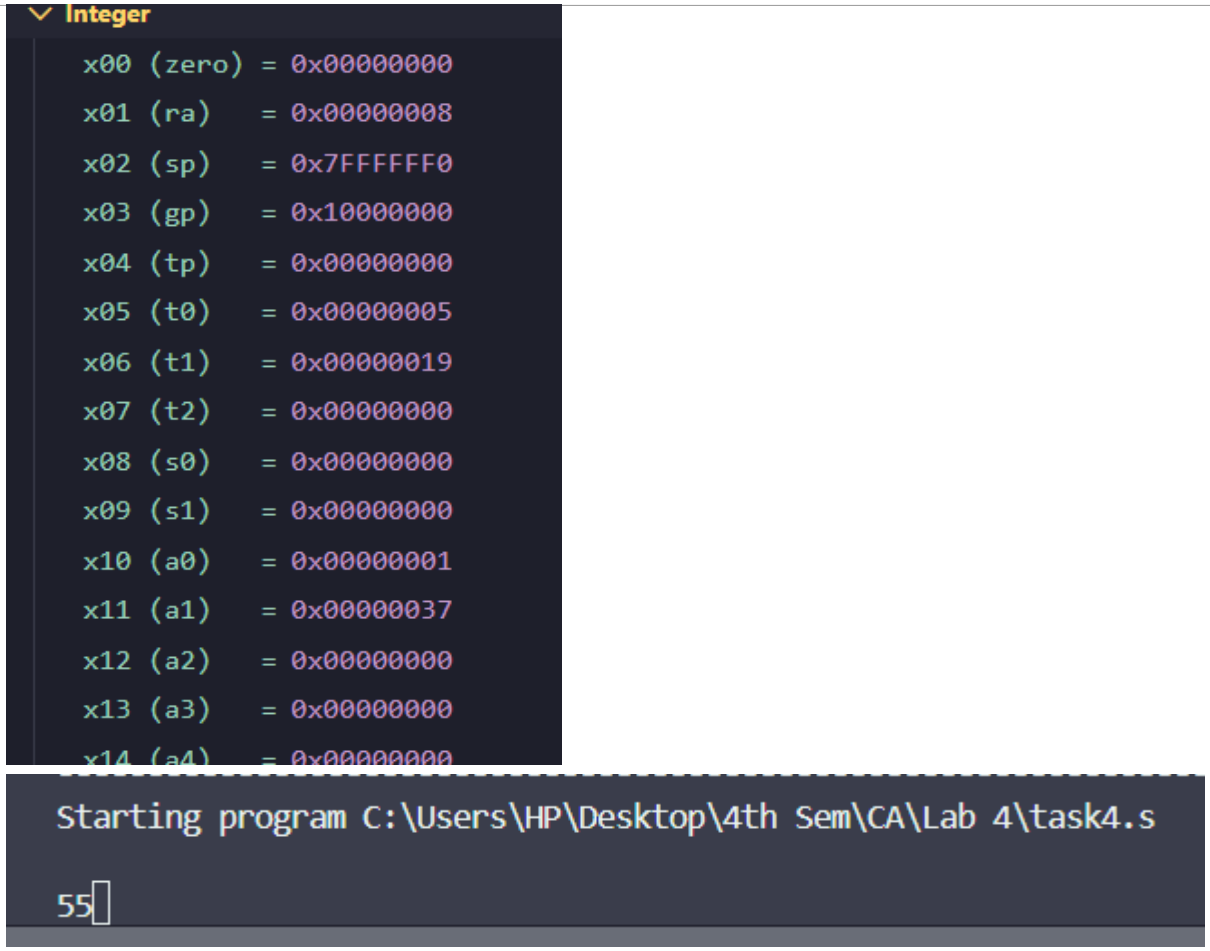
Students are encouraged to select a program that can be extended or reused in their project. Proper use of procedure calls, register conventions, and stack discipline will be assessed. Students should clearly demonstrate correct execution of the program.

Code:

task4.s

```
.globl main
.text
main:
    addi x10, x0, 5    # n = 5
    jal x1, sum_square

    addi x11, x10, 0    #Resultto x11
    addi x10, x0, 1      # Print Integer
    ecall
    addi x10, x0, 10
    ecall

sum_square:
    addi x2, x2, -8
    sw x1, 4(x2)
    sw x10, 0(x2)

    beq x10, x0, basecase
    addi x10, x10, -1    # n = n - 1
    jal x1, sum_square     #Recursive call

    lw x5, 0(x2)        #Restore original n from stack to x5
    mul x6, x5, x5     # x6 = n*n
    add x10, x10, x6    # add result to n*n

    jal x0, exit_sum

basecase:
    addi x10, x0, 0 # Return 0

exit_sum:
    lw x1, 4(x2)    # Restore return address
    addi x2, x2, 8      # Restore stack pointer
    jalr x0, 0(x1)
```

Output


Integer
x00 (zero) = 0x00000000
x01 (ra) = 0x00000008
x02 (sp) = 0x7FFFFFF0
x03 (gp) = 0x10000000
x04 (tp) = 0x00000000
x05 (t0) = 0x00000005
x06 (t1) = 0x00000019
x07 (t2) = 0x00000000
x08 (s0) = 0x00000000
x09 (s1) = 0x00000000
x10 (a0) = 0x00000001
x11 (a1) = 0x00000037
x12 (a2) = 0x00000000
x13 (a3) = 0x00000000
Starting program C:\Users\HP\Desktop\4th Sem\CA\Lab 4\task4.s
55

Assessment Rubric
Lab 4: Implementing Nested Procedures and Sorting

Name: M. Abdullah Asim, Maaz Ahmed Siddiqui	Student ID: ma10020, ms09980	Section: T3

Points Distribution

	Task No.	LR 2 Code	LR 5 Results
In - Lab	Task 1	/10	/5
	Task 2	/10	/5
	Task 3	/15	/10
	Task 4	/15	/10
Total Points: 100		**/50**	**/30**
CLO Mapped		CLO 2	

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission / Git Upload	/10 & /10	CLO 2

CLO	Total Points	Points Obtained
2	100	
Total	**100**	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.